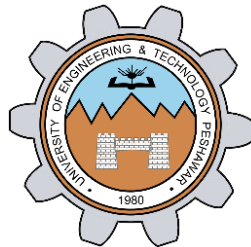


Lab# 03: **Boosting OOP Proficiency: Exploring Static Components & Object Relationships**



Fall 2025

Submitted by
M Mohsin Khan (24pwcse2382)

Class Section: C

"On my honor, as student of University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work."

Submitted to: **Engr. Sumayyah Salahuddin**

Oct 24th, 2025

Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar

Objectives of the Lab

1. Understand how static class components differ from non-static ones in terms of memory allocation and scope of functionality.
2. Examine how C++ const members and objects maintain data integrity, enforce immutability, and influence program behaviour.
3. Compare objects and pointers in C++, focusing on their roles in function execution, memory management, and the complexity of embedded objects.
4. Design and execute test cases to demonstrate a clear understanding of static components, const members, objects, and pointers in C++.

Activity # 3.4.1: Employee Database Management System

Problem Analysis:

Design a class named Employee to represent an employee in a company. The class should include the following features:

A static integer variable totalEmployees to keep track of the total number of employee objects created.

Non-static data members:

- a. employeeId (int) to store the employee's unique identification number.
- b. name (character array of size 50) to store the employee's name.
- c. designation (character array of size 50) to store the employee's job title or position.

A constructor that initializes the employee's ID, name, and designation, while incrementing totalEmployees each time a new object is created.

Member functions to:

- a. Add new employee details.
- b. Display employee information.
- c. Update existing employee data such as name or designation.

A static member function to display the total number of employees currently in the system.

Demonstrate both static and non-static features by creating and modifying multiple employee objects within the main() function.

Algorithm

- Create a class named Employee.
- Declare a static integer variable totalEmployees to maintain the count of employees.
- Declare non-static members: employeeId, name[50], and designation[50].
- Define a constructor to initialize employee details and increment the totalEmployees variable.
- Define addEmployee() to input employee data.
- Define showEmployeeRecord() to show employee information.
- Define updateEmployeeRecord() to edit existing employee details.
- Define a static showRTotalEmployees() function to show the total number of employees created.
- In the main() function, create multiple Employee objects, use the defined functions to add, display, and update records, and show the total number of employees to demonstrate the distinction between static and non-static behaviour.

Hand written Code:

Day: _____ Date: _____

LAB 3 TASK : 1

```
#include <iostream>
#include <string>
using namespace std;

class Employee {
    static int totalEmployees;
    int employeeId;
    char name[50];
    char designation[50];

public:
    Employee(int id, const char empName,
             const char empDesignation[]) {
        employeeId = id;

        strcpy(name, empName);
        strcpy(designation, empDesignation);
        totalEmployee++;
    }

    void displayInfo() const {
        cout << "Employee ID: " << employeeId << endl;
        cout << "Name: " << name << endl;
        cout << "Designation: " << designation << endl;
    }
}
```

Day: _____ Date: _____

```
void updateDesignation(const char
                       newDesignation[]) {
    strcpy(designation, newDesignation);
    cout << "Employee" << employeeId;

    static void displayTotalEmployees()
    {
        cout << "Total Employees: " << totalEmployees
              << endl;
    }
};
int Employee::totalEmployees = 0;

int main() {
    Employee::displayTotalEmployees();

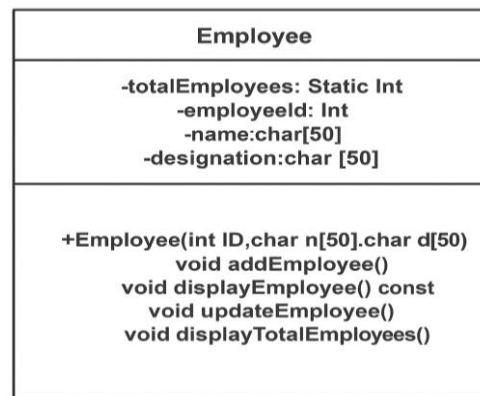
    Employee emp1(1, "Abdullah", "Manager");
    emp1.displayInfo();

    Employee emp2(2, "Sana", "Developer");
    emp2.displayInfo();

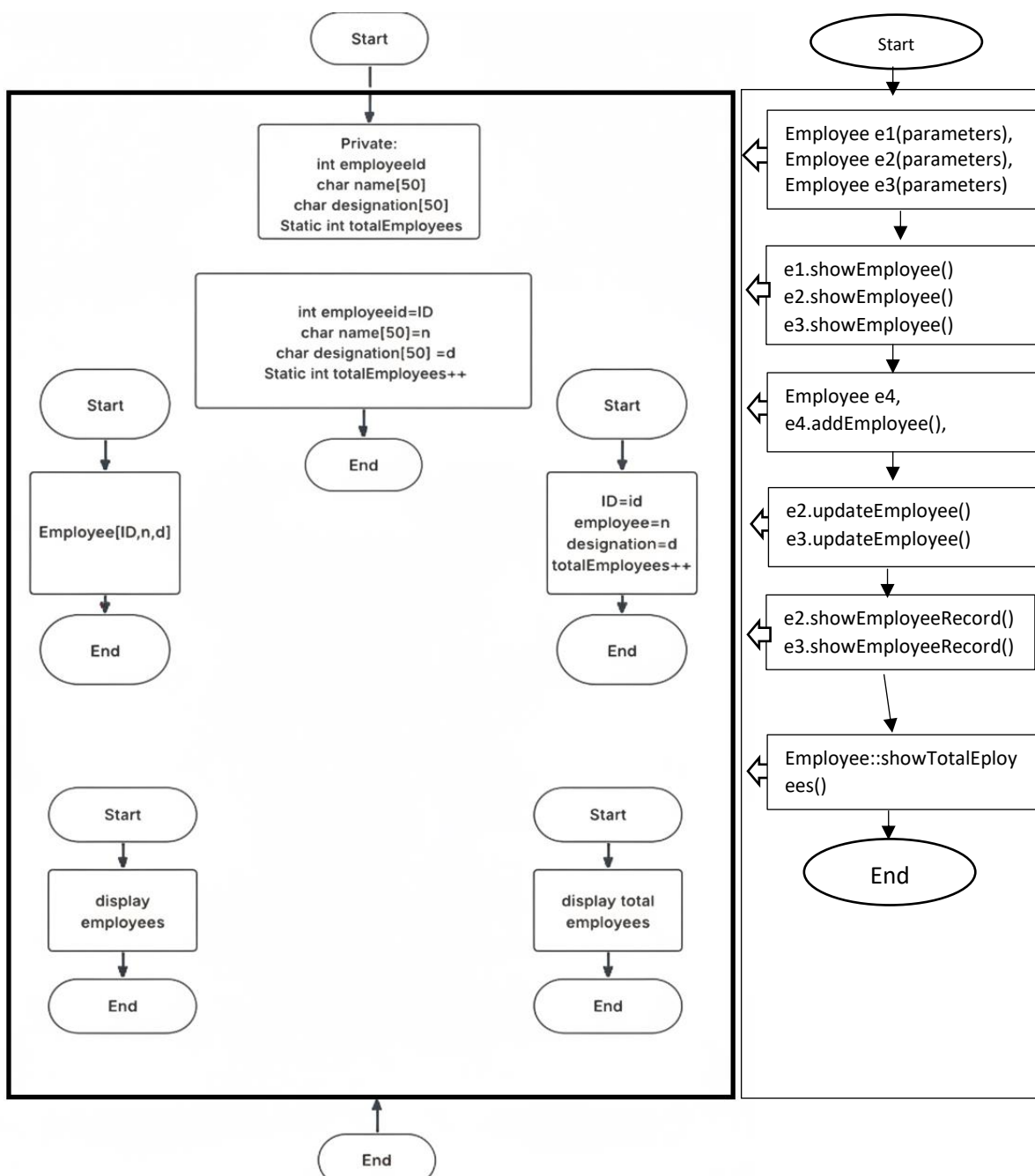
    emp2.updateDesignation("Senior Developer");
    emp2.displayInfo();

    Employee::displayTotalEmployees();
    return 0;
}
```

2.5.1 ML Diagram:



2.5.2 Flow Chart



2.5.1 Source Code:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Employee
5  {
6  private:
7      static int totalEmployees;
8      int employeeId;
9      char name[50], designation[50];
10
11 public:
12     Employee() : employeeId(0)
13     {
14         strcpy(name, "Null");
15         strcpy(designation, "Null");
16     }
17
18     Employee(const char n[], const char d[], optional<int> id = nullopt)
19     {
20         employeeId = id.has_value() ? id.value() : ++totalEmployees;
21         strcpy(name, n);
22         strcpy(designation, d);
23         cout << "Employee created: " << name << " (ID: " << employeeId << ")\n";
24     }
25
26     void addEmployee()
27     {
28
29         cout << "\nPress Enter to add new Employee to Database ";
30
31         cin.ignore();
32         cout << "Enter name: ";
33         cin.getline(name, 50);
34         cout << "Enter designation: ";
35         cin.getline(designation, 50);
36         employeeId = ++totalEmployees;
37         cout << "Employee " << name << " added with ID: " << employeeId << "\n";
38     }
39
40     void updateEmployeeRecord()
41     {
42         cout << "\nUpdate " << name << "'s record: (ID: " << employeeId << ")\n";
43         cout << "1-> Name: " << name << "\n2-> Designation: " << designation << "\n3-> ID: ";
44         int choice;
45         cin >> choice;
46         cin.ignore();
47
48         switch (choice)
49         {
50             case 1:
51                 cout << "Enter new name: ";
52                 cin.getline(name, 50);
53                 break;
54             case 2:
55                 cout << "Enter new designation: ";
56                 cin.getline(designation, 50);
57                 break;
58             case 3:
59
60                 cout << "Enter new ID: ";
61                 cin >> employeeId;
62                 break;
63             default:
64                 cout << " Invalid choice.\n";
65                 return;
66         }
67         cout << "Employee record updated successfully.\n";
68     }
69
70     void showEmployeeRecord() const
71     {
72         cout << "\nEmployee Name : " << name << "\n";
73         cout << "Designation : " << designation << "\n";
74         cout << "Employee ID : " << employeeId << "\n";
75     }
76
77     static void showTotalEmployees()
78     {
79         cout << "\nTotal Employees in System: " << totalEmployees << "\n";
80     }
81 };
82
83 int Employee::totalEmployees = 0;
84
85 int main()
86 {
87
88     Employee e1("Ahmed", "Manager");
89     Employee e2("Ali", "Developer");
90     Employee e3("Sara", "HR", 101);
91
92     e1.showEmployeeRecord();
93     e2.showEmployeeRecord();
94     e3.showEmployeeRecord();
95
96     Employee e4;
97     e4.addEmployee();
98     e4.showEmployeeRecord();
99
100    e1.updateEmployeeRecord();
101    e2.updateEmployeeRecord();
102
103    e1.showEmployeeRecord();
104    e2.showEmployeeRecord();
105
106    Employee::showTotalEmployees();
107 }
```

Output:

```
Employee created: Ahmed (ID: 1)
Employee created: Ali (ID: 2)
Employee created: Sara (ID: 101)

Employee Name : Ahmed
Designation   : Manager
Employee ID   : 1

Employee Name : Ali
Designation   : Developer
Employee ID   : 2

Employee Name : Sara
Designation   : HR
Employee ID   : 101

Press Enter to add new Employee to Database
Enter name: Mohsin Khan
Enter designation: Developer
Employee 'Mohsin Khan' added with ID: 3

Employee Name : Mohsin Khan
Designation   : Developer
Employee ID   : 3

Update Ahmed's record; (ID: 1)
1 -> Name: Ahmed
2 -> Designation: Manager
3 -> ID: 1
Select field to update: 2
Enter new designation: Web lead
Employee record updated successfully.

Update Ali's record; (ID: 2)
1 -> Name: Ali
2 -> Designation: Developer
3 -> ID: 2
Select field to update: 2
Enter new designation: AI co-lead
Employee record updated successfully.

Employee Name : Ahmed
Designation   : Web lead
Employee ID   : 1

Employee Name : Ali
Designation   : AI co-lead
Employee ID   : 2

Total Employees in System: 3
```

3.4.2 Activity: Bank Account Management System

1. Design a class BankAccount to represent a bank account, incorporating const members to store the account holder's personal details. Ensure that these details remain unchanged throughout the account's lifetime.
2. Write functions to deposit and withdraw funds, along with a method to display the account holder's information.
3. Illustrate the practical use of const members and objects within the banking context.

Hand Written Source Code:

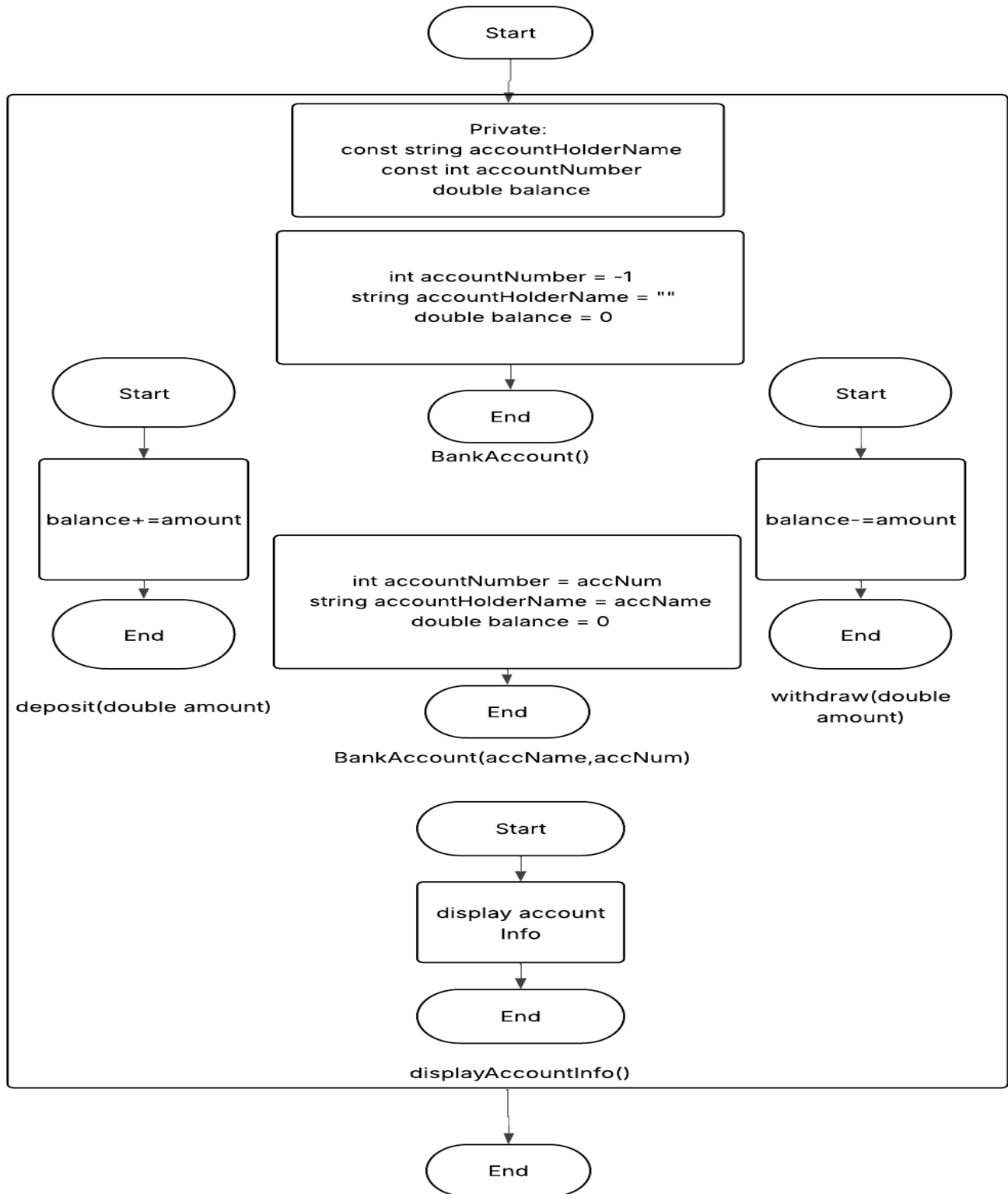
Date: _____ Day: _____

Lab #03
Bank Management System

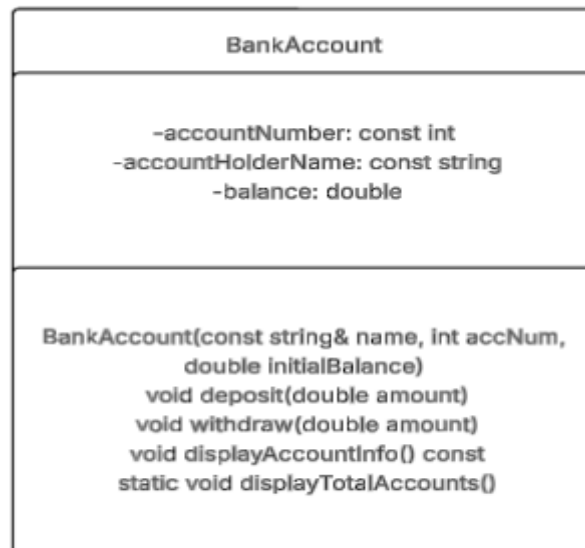
```
#include <iostream>
using namespace std;
class BankAccount {
private:
    const string name, accNo;
    double balance;
public:
    BankAccount(string n, string a, double b = 0.0) :
        name(n), accNo(a), balance(b) {}
    void deposit(double amt) {
        balance += amt;
        cout << "[+]" << amt << " deposited.\n";
    }
    void withdraw(double amt) {
        if (amt > balance)
            cout << "[-] insufficient funds!\n";
        else
            balance -= amt, cout << "[-]" << amt << " withdrawn!\n";
    }
    void show() const {
        cout << "\n Account No : " << accNo << " Balance: " <<
            balance << "\n";
    }
};

int main() {
    BankAccount acc("Ahmed Khan", "PK 12345", 500);
    acc.show();
    acc.deposit(1500);
    acc.withdraw(2000);
    acc.show();
}
```

Flow Chart:



UML Diagram:



Source Code:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class BankAccount
5  {
6      const string name, accNo;
7      double balance;
8
9  public:
10     BankAccount(string n, string a, double b = 0.0) : name(n), accNo(a), balance(b) {}
11     void deposit(double amt)
12     {
13         balance += amt;
14         cout << "[+]" << amt << " deposited.\n";
15     }
16     void withdraw(double amt)
17     {
18         if (amt > balance)
19             cout << "[-] Insufficient funds!\n";
20         else
21             balance -= amt, cout << "[-]" << amt << " withdrawn.\n";
22     }
23     void show() const
24     {
25         cout << "\n--- Account Info ---\nName: " << name
26             << "\nAccount No: " << accNo
27             << "\nBalance: " << balance << "\n";
28     }
29 };
30
31 int main()
32 {
33     BankAccount acc("Ahmed Khan", "PK12345", 5000);
34     acc.show();
35     acc.deposit(1500);
36     acc.withdraw(2000);
37     acc.show();
38 }
39
```

Output:

```
--- Account Info ---
Name: Ahmed Khan
Account No: PK12345
Balance: 5000
[+]1500 deposited.
[-]2000 withdrawn.

--- Account Info ---
Name: Ahmed Khan
Account No: PK12345
Balance: 4500
```